Thesis Structure: Hierarchical Reinforcement Learning for Multi-Asset Trading

Topic: Separating Temporal Extraction from Cross-Sectional Allocation in DRL for portfolio allocation problems
Environment: Custom coded env called PyTrade

1. Introduction

- **1.1 Motivation:** Financial markets represent highly volatile and complex decision-making environments in artificial intelligence. Research in this domain explores the boundaries of how reinforcement learning agents handle extreme noise, non-stationarity, and multi-layered problem spaces. Then proceed to explain: Why buying and holding ETFs might not be the most efficient strategy with the given amount of data and tools available. Transitioning from static forecasting to sequential decision-making in non-stationary markets. Why DRL is good at sequential decisions.
- **1.2 Problem Description:** The challenge of high-dimensional continuous action spaces and the low signal-to-noise ratio in financial data.
- **1.3 Research Question:** Does modularizing temporal (Single-Asset) and cross-sectional (Portfolio-Level) functions improve stability and efficiency?

2. Literature Review

- **2.1 DRL in Quantitative Finance:** Evolution from DQN to PPO in portfolio management.
- **2.2 Temporal Memory in Financial Series:** Comparison of RNNs, LSTMs, and TCNs for state representation.
- **2.3 Attention Mechanisms:** The shift toward modeling time-varying dependencies across assets.
- **2.4 Hierarchical & Modular RL:** Theoretical basis for decomposing complex policies into specialized modules.

3. Mathematical Framework & MDP Formulation

- **3.1 Markov Decision Process (MDP):** Formal definition of State $\mathcal{S}$, Action $\mathcal{A}$, Transition $\mathcal{P}$, and Reward $\mathcal{R}$.
- **3.2 State Space Representation:** Mathematical derivation of stationary features from raw OHLCV. Introduction of memory preserving features (Lopez and de Prado). Why just adding more data is not helpful and finding the right input data is so hard.
- **3.3 Action Space:** For PAA: continuous portfolio weight targets $w \in \Delta^n$, executed in enclosing steps by the env based on the real current portfolio weights. For the SAA module, the action is a single scalar which is defined as the requested change in position relative to the current portfolio notional.
- **3.4 Reward Shaping for Single Asset Agent (SAA):**
    - **3.4.1 Multi-Objective Reward Composition:** Decomposing rewards into the "Nested Box" framework. Learning the hard limitations within a portfolio first (execution gap), followed by generating returns (beta), and finally generating alpha within the single instrument. Basically curriculum learning.

    - **3.4.2 The Execution Gap:** Mathematical formulation of penalties for requested actions exceeding available cash or portfolio constraints.
    - **3.4.3 Risk-Adjusted Alpha:** Use of SAA Excess Log Return vs simple all-hold baseline (alpha) and Differential Sortino ratios (volatility vs returns) as the core performance signals. The main question to answer here is how not to reward the agent with market beta, since markets on a large scale have generally been moving up.

4. Proposed Hierarchical Architecture

- **4.1 Layer 1: Temporal Extraction Module (Single Asset Agent):**
    - **4.1.1 Dual-Recursion Logic:** Utilizing sb3-contrib RecurrentPPO to manage temporal state transitions alongside a 2-layer LSTM architecture for hierarchical feature extraction (Noise filtering through LSTM vs. Regime detection through recurrent algorithm). The SAA is trained on randomly rotating assets but with a learned small asset-id embedding. In later use the single trained agent is copied per asset. This is necessary due to the statefulness of recurrent agents.
    - **4.1.2 Training Dynamics and Stability:** Analysis of hyperparameter sensitivity, specifically the relationship between high Entropy (exploration) and Learning Rate decay in preventing policy collapse (Long-only/Short-only traps). A key architectural challenge in modular DRL frameworks is avoiding the transition dynamics mismatch when moving from single-agent training to multi-agent deployment. If the temporal extractor (Layer 1) is exposed to global portfolio states like shared cash, this is fine in individual training. But when moving to wiring the individual agents together their recurrent memory (LSTM) becomes susceptible to non-stationary feedback loops caused by exogenous agent actions. This is essentially a domain shift: changing the environment in a way that was not seen during training. The solution for this is adressed in layer 2 description (4.2).
- **4.2 Layer 2: Cross-Sectional Allocator (Portfolio Allocator Agent):** Self-attention mechanisms to coordinate the extracted features into a portfolio. Uses the output of frozen SAAs as features. The individual SAA agents operate in shadow portfolios, that do not have actual influence on the portfolio allocations made by the PAA. They work as indipendent advisors on the individual asset. Data ingestion happens through a portfolio token and n asset tokens. this design enables clean self-attention across assets. Below is a graph indicating the token structure for an asset token.

PAA INPUT VECTOR (Asset i)

1. Raw Market Context	**2. Local Shadow State**	**3. Latent Memory**
(rsi_14, z_market_dispersion, etc.)	(Shadow Weight, SAA Action)	(64D LSTM Hidden St.)

Provides PAA (layer 2) with the Problem (Market Features), the Proposed Local Solution (SAA Action), and the Confidence/Context Matrix (Hidden State). One risk remains which is feature dillution. This may be mitigated by the networks architecture, or exploring whether to leave out the latent memory, as it is quite large.

- **4.3 Information Flow:** How the temporal embeddings are concatenated and passed to the attention head.
  Framing: Empirical test of Feature Information Density. I am testing whether a neural network can allocate assets more efficiently when it receives a combination of raw unstructured market data and structured behavioral embeddings generated by an isolated temporal expert. If the PAA outperforms a model trained only on raw features, I have academic proof that the hierarchical decomposition works.

5. The PyTrade Environment & Implementation

- **5.1 Simulator Design:** Building a Gymnasium-compliant environment for experimental control.
- **5.2 Friction Modeling:** Mathematical implementation of commissions, spreads, and market impact. Logical proof via transaction cost verification graphs.
- **5.3 Asset Universe:** Selection of the 11 instruments (2000-2025) and the rationale for their diversity including SPY, Gold, Oil, and international indices (EWG, EWQ).

6. Verification

- **6.1 Out-of-Sample (OOS) Protocol:** Walk-forward testing on unseen data blocks to distinguish true Alpha from memorized noise.
- **6.2 SAA Performance Baselines:**
    - **6.2.1 Cash-Only Baseline:** Comparing individual agent returns against holding only cash.
    - **6.2.2 Buy & Hold Baseline:** Comparing individual agents against passive buy-and-hold performance for the same asset.
    - **6.2.3 50% Invested Baseline:** Comparing agent performance against a half-invested static exposure.
- **6.3 Statistical Robustness SAA:** Discussing the statistical relevance of the found alpha, identifying the "Peak Alpha", and analyzing validation decay as evidence of overfitting.
- **6.4 PAA Performance Baselines:**
    - **6.2.1 Cash-Only Baseline:** Comparing PAA returns against holding only cash.
    - **6.2.2 Buy & Hold Baseline:** Comparing PAA returns against passive unfirmly spread buy-and-hold performance across all assets.
    - **6.2.3 50% Invested Baseline:** Comparing agent performance against a half-invested static exposure.
    - **6.2.4 Industry-grade allocation Buy & Hold Baseline:** Comparing agent performance against a predefined allocation weight portfolio (e.g. 45% SPY, 10% Gold, etc.)
- **6.5 Statistical Robustness PAA:** Discussing the statistical relevance of the found alpha, identifying the "Peak Alpha", and analyzing validation decay as evidence of overfitting.

7. Proposed Experiment Tests

- **7.1 Baseline (Cross-sectional only):** A Transformer-based portfolio allocator utilizing only same-day features, without augmented temporal memory.
- **7.2 Hierarchical (Modular memory):** The proposed two-level allocator where cross-sectional attention is augmented by pre-extracted recurrent temporal signals.
- **7.3 Control (Randomized signal):** The hierarchical setup but with the SAA signals replaced by structured noise, testing whether performance gains stem from actual historical information rather than mere architectural complexity.
- **7.4 Monolithic Baseline (End-to-end memory):** A standard recurrent PPO allocator that must learn both temporal memory and cross-asset allocation simultaneously. Might introduce a 2-layer LSTM to the PAA before or after transformer?

8. Conclusion & Future Work

- **8.1 Summary of Findings:** Validating the hierarchical approach for Financial RL and the success of "Box-based" reward shaping in stabilizing SAA agents.

- **8.2 Limitations:** Data frequency constraints, the impact of zero-friction training assumptions, and the "sim-to-real" gap.
- **8.3 Future Directions:** Multi-agent extensions for PAA coordination or integrating alternative data into the SAA temporal extraction layer.